OmniMind Agent: Central Inteligente Transmídia

Agente Multimodal Sem Fricção de Captura & Organização Pessoal Cross-Device

Evento: All Things Agentic Hackathon (Google) **Trilha:** Collaborative Partner / Taskmaster **Prazo Final:** 31 de Agosto de 2026

1. Visão Geral e Proposta de Valor

O **OmniMind Agent** é uma solução de inteligência pessoal projetada para eliminar o atrito de captura de informações no dia a dia. Atualmente, usuários utilizam o próprio aplicativo de mensagens (como conversas consigo mesmo no WhatsApp ou Telegram) para salvar áudios soltos, fotos de comprovantes, links para leitura posterior e ideias rápidas. No entanto, esses dados tornam-se rapidamente um "caos não estruturado", impossível de pesquisar e organizar.

> **Conceito Central:** O usuário envia qualquer tipo de mídia ou texto via **Telegram Bot** (acessível em tempo real no smartphone ou desktop). O **Agente Gemini** processa o conteúdo nos bastidores de forma autônoma, categoriza, extrai ações e armazena os dados de forma estruturada no **Google Cloud Firestore**. Os dados ficam instantaneamente pesquisáveis e visualizáveis em um **Dashboard Streamlit**.

2. Arquitetura da Solução e Ecossistema Google Cloud

A arquitetura foi projetada para combinar alta velocidade de desenvolvimento (essencial para o hackathon de 6 dias), custo zero de infraestrutura no modo de degustação e integração nativa com o ecossistema do Google Cloud.

Camada Architectural	Tecnologia Escolhida	Papel no Sistema & Justificativa Técnica
Ponto de Captura (Input)	`Telegram Bot API`	Interface de captura sem fricção nativa no iOS, Android, Mac e Windows. Permite envio imediato de áudio, voz, imagem, PDF, arquivos e links. Setup em 2 minutos via `@BotFather`.
Motor Agêntico (IA)	`Gemini API (2.0 / 1.5)`	Processamento multimodal nativo (visão computacional + áudio direto sem necessidade de Whisper separado) com saídas estruturadas via Pydantic JSON e *Function Calling*.
Banco de Dados	`Google Cloud Firestore`	Banco de dados NoSQL Serverless altamente escalável. Cota gratuita robusta, sincronização em tempo real e facilidade de integração com o SDK oficial `google-cloud-firestore`.
Armazenamento de Mídia	`Google Cloud Storage (GCS)`	Armazenamento de buckets para salvar os áudios originais, imagens capturadas e documentos recebidos para consulta futura.
Visualização & Busca	`Streamlit Dashboard`	Interface web reativa e responsiva para celular e PC. Exibe busca semântica, categorização por tags, lista de tarefas extraídas e filtros de mídia.
Hospedagem & Nuvem	`Google Cloud Run`	Deploy da aplicação encapsulada em container Docker. Operação no *Always Free Tier* (até 2 milhões de requisições/mês sem custo).

Diagrama do Fluxo de Dados

```
[Dispositivo Usuário] → Envia Áudio/Foto/Link/Texto via Telegram Bot
         |
         ▼
[Python Backend / Bot Handler] → Salva mídia temporária no Google Cloud Storage (GCS)
         |
         ▼
[Agente Gemini 2.0 / 1.5 Flash] → Analisa Mídia + Prompt Agêntico + Pydantic Schema
         |                        ↳ Extrai: Resumo, Categoria, Tags, Tarefas, Ações
         ▼
[Google Cloud Firestore] → Persiste o documento JSON Estruturado
         |
         ▼
[Streamlit Dashboard Web] → Interface de Busca Semântica, Painel Kanban e Linha do Tempo
```

3. Fluxos Práticos de Utilização e Resultados Esperados

Abaixo estão os 4 cenários principais que serão demonstrados no vídeo de submissão do hackathon, evidenciando o poder agêntico do Gemini em transformar caos em estruturação.

Cenário A: Nota de Voz em Trânsito (Áudio Multimodal)

Entrada do Usuário: Grava um áudio de 18 segundos no Telegram dizendo: *"Preciso lembrar de ligar para o fornecedor de peças amanhã às 14h para cobrar o orçamento da revisão do carro, e também comprar café para o escritório."*

Ação do Agente Gemini: O Gemini consome o arquivo `.ogg` diretamente. Transcreve com alta precisão, identifica intenções e gera 2 tarefas acionáveis com datas.

Resultado Estruturado (Firestore & Dashboard):

Categoria: Lembretes / Tarefas | Tag: #ManutençãoCarro | Tag: #Escritório

Tarefas Extraídas:

- [ ] Ligar para fornecedor de peças - Lembrete: Amanhã às 14:00.
- [ ] Comprar café para o escritório.

Cenário B: Foto de Comprovante ou Recibo (Visão Computacional)

Entrada do Usuário: Tira foto de uma nota fiscal de restaurante ou recibo de estacionamento e manda no Telegram sem nenhum texto adicional.

Ação do Agente Gemini: O Gemini realiza OCR e parsing visual, identificando estabelecimento, CNPJ/Data, itens e valor total gasto.

Resultado Estruturado (Firestore & Dashboard):

Categoria: Finanças / Gastos | Estabelecimento: Outback Steakhouse

Dados Financeiros: Valor: `R$ 184,50` | Data: `25/08/2026` | Tipo: Alimentação.

Relatório no Dashboard: Atualiza automaticamente o gráfico diário de despesas.

Cenário C: Envios de Links e Artigos da Web

Entrada do Usuário: Envia a URL de um artigo técnico longo sobre arquitetura de software ou notícias de mercado.

Ação do Agente Gemini: Faz o web scraping do conteúdo da página, gera um resumo executivo em 3 bullet points e classifica o nível de complexidade do tema.

Resultado Estruturado (Firestore & Dashboard):

Categoria: Base de Conhecimento Tag: #Tecnologia #IA

Resumo Agêntico:

- Ponto 1: Explica a transição de microsserviços para arquitetura baseada em agentes.
- Ponto 2: Destaca redução de custo computacional em 35%.
- Ponto 3: Principais bibliotecas recomendadas para implementação.

Cenário D: Brain Dump / Texto Bagunçado em Bloco Único

Entrada do Usuário: Envia um bloco de texto contínuo e sem pontuação com várias ideias misturadas.

Ação do Agente Gemini: Desmembra o texto, separa o que é projeto, o que é compra pessoal e o que é insight criativo, alocando nas respectivas pastas virtuais.

4. Especificação Técnica do Schema de Resposta (Pydantic)

Para garantir que a saída do Gemini seja 100% confiável e pronta para gravação no Firestore, o agente utiliza o modelo de resposta estruturada nativo do SDK unificado `google-genai`:

```
from pydantic import BaseModel, Field
from typing import List, Optional

class TaskItem(BaseModel):
    description: str = Field(description="Descrição clara da ação ou tarefa")
    due_date: Optional[str] = Field(description="Data/hora de vencimento ISO format, se mencionada")
    priority: str = Field(description="Alta, Média ou Baixa")

class ProcessedContentSchema(BaseModel):
    summary: str = Field(description="Resumo executivo do conteúdo processado em 1-2 frases")
    category: str = Field(description="Finanças, Tarefas, Conhecimento, Saúde, Lembretes, Ideias")
    tags: List[str] = Field(description="Lista de tags semânticas para busca avançada")
    tasks: List[TaskItem] = Field(default_factory=list, description="Lista de tarefas acionáveis extraídas")
    financial_amount: Optional[float] = Field(description="Valor monetário extraído, caso seja um comprovante/
despesa")
    key_insights: List[str] = Field(default_factory=list, description="Pontos chave do conteúdo")
```

5. Estratégia do Projeto para Vencimento de Categoria

Diferenciais Competitivos Avaliados pelos Juízes

- **Multimodalidade Real:** Uso direto de áudio e imagem no Gemini sem intermediários pesados.
- **Solução de Dor Real:** Resolve a desorganização diária de milhões de pessoas que usam chats como bloco de notas.
- **Integração Google Cloud Native:** Firestore + Cloud Storage + Cloud Run + Gemini API.
- **Interface Dupla:** Praticidade do Telegram no mobile/desktop + elegância do Streamlit para analytics.

Roteiro Recomendado para o Vídeo de 3 Minutos

- **0:00 - 0:30 (O Problema):** Mostrar o caos de conversas no WhatsApp/Telegram cheias de áudios e printouts.
- **0:30 - 1:45 (A Solução em Ação):** Enviar áudio e nota fiscal ao vivo no Telegram e mostrar o bot processando.
- **1:45 - 2:30 (O Dashboard):** Abrir o Streamlit no PC, fazer busca semântica e mostrar a lista de tarefas gerada.
- **2:30 - 3:00 (Arquitetura):** Exibir o diagrama do Google Cloud Run + Firestore e concluir com o pitch de valor.

6. Plano de Execução Rápida em 6 Dias (Até 31/08/2026)

Dia	Foco Principal de Desenvolvimento	Entregável Concreto
Dia 1 (25/08)	Criação do Bot no Telegram (`@BotFather`) + Configuração da API do Gemini (AI Studio).	Script Python recebendo mensagens de texto e áudio no Telegram.
Dia 2 (26/08)	Implementação dos Handlers Multimodais (Gemini + Pydantic Schema) e integração Firestore.	Envio de mídia salvando JSON estruturado no Google Cloud Firestore.
Dia 3 (27/08)	Desenvolvimento da Interface Web em Streamlit (Busca semântica + Filtro por Tags + Tarefas).	Dashboard operacional e integrado ao banco de dados Firestore.
Dia 4 (28/08)	Empacotamento Docker + Deploy da aplicação no **Google Cloud Run**.	URL pública ativa da aplicação rodando na nuvem do Google Cloud.
Dia 5 (29/08)	Gravação e edição do vídeo demonstrativo de 2.5 minutos para o Devpost.	Vídeo editado e publicado no YouTube/Vimeo.
Dia 6 (30-31/08)	Documentação do repositório GitHub + Preenchimento final do formulário no Devpost.	Submissão do projeto concluída com sucesso!